

AirBreda: Architecture Design Document

Author: Michael Favour Eriabie (236474), BUas ADSAI Year 3, System Design & Cloud Platforms

Question: Is traffic on the A27 near Breda linked to NO₂ at Luchtmeetnet station NL10240 (Breda-Tilburgseweg)?

Live system: `http://16.171.236.39:8000 (/, /site/{site_id}, /health)`

Date: 3 October 2026

1. Architecture as deployed

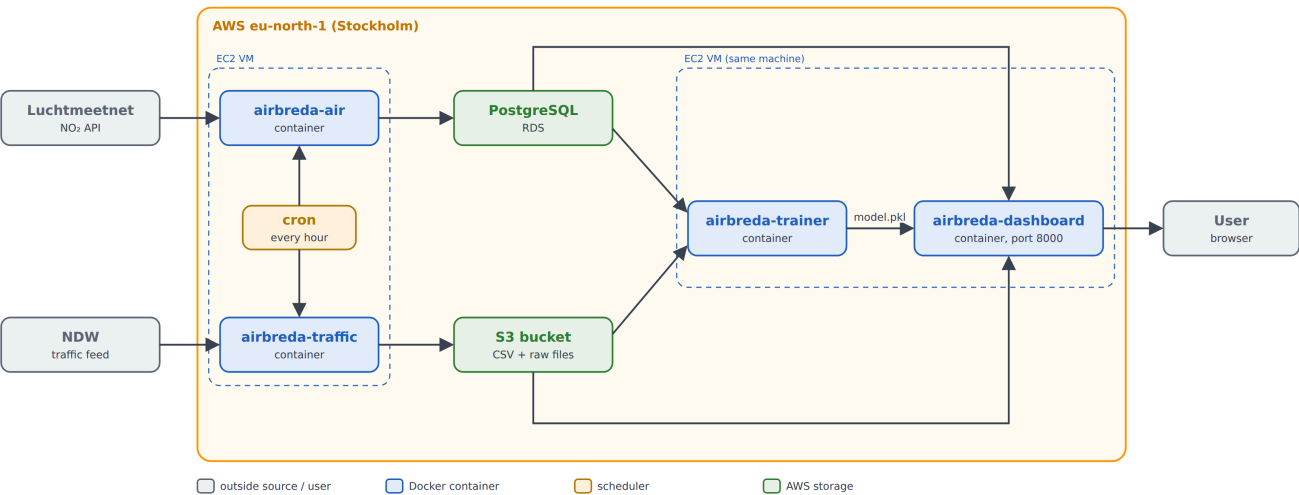


Figure 1. How data flows through AirBreda on AWS eu-north-1 (Stockholm).

I run everything on one small EC2 virtual machine. Every hour, cron starts two containers. The first fetches NO₂ from Luchtmeetnet and writes it to PostgreSQL on RDS. The second downloads the national NDW traffic file, keeps my four A27 sensors, and saves one CSV per sensor plus the raw file to S3. A third container, the dashboard, runs permanently on port 8000. It reads both stores, applies the model and serves the page and the API. I train the model by hand with a separate trainer image.

Trust boundaries

Boundary	Who can cross it	How I enforce it
Internet → VM port 8000	anyone	security group, 0.0.0.0/0 on port 8000
Internet → VM port 22 (SSH)	only my current IP	security group, source = My IP
VM → database port 5432	only the VM	security groups created by "Connect to an EC2 compute resource"; the database is not public
VM → S3 bucket	only the VM's role, only this bucket	IAM role with a least-privilege policy
Secrets	only on the VM	.env (chmod 600), in .gitignore, never in Git or an image

The Day 2 Redis queue is not deployed (see ADR-002 and ADR-004). docker-compose is for local testing only.

2. Architecture Decision Records

ADR-001: Data storage

Status: Accepted (Day 1)

Context. Luchtmeetnet gives one NO₂ value per hour as JSON, and every call returns the last 50 hours again, so duplicates are guaranteed. NDW publishes one national XML file every minute (about 1.2 MB zipped, 20,000 sites), of which I need four sites. I need time-range queries, a way to match both sources by hour, and protection against duplicates.

Options. PostgreSQL plus S3; DynamoDB for everything; or a database only, throwing the raw XML away.

Decision. NO₂ goes into **PostgreSQL** (sensor_readings, primary key station_id, timestamp, component). Traffic goes into **S3**: one CSV per sensor per hour (ndw/YYYY-MM-DD/HH-site.csv) and the untouched raw file (raw/ndw/YYYY-MM-DD/HH.xml.gz). Writes use INSERT ... ON CONFLICT DO NOTHING.

Consequences. - NO₂ is small, structured rows that I query, which is what a relational database is for. DynamoDB's scaling solves a problem I don't have. - Duplicates are harmless. A normal hourly run logs fetched: 50, inserted: 1, skipped_existing: 49. The fetch is at-least-once and the write is idempotent, so the result behaves like exactly-once. - Keeping the raw XML means I can re-parse everything if I find a bug in my parsing. Throwing it away would make such a bug permanent. - Raw files cost about 0.85 GB a month, roughly \$0.02. - Weak spot: the dashboard has to list the bucket to find the newest traffic files. Fine for four sensors, not for 50 corridors.

ADR-002: Messaging

Status: Superseded by ADR-004. I keep it so the reasoning stays visible.

Context. Day 2 introduced event-driven design: producers publish to a broker and consumers subscribe, which makes it easy to add consumers later.

Decision (Day 2). Both collectors published each reading as JSON to a Redis list (docker-compose.day2.yml, local only). I picked Redis because it is one container with no cloud setup. In production I would use SQS.

Broker failure. I tested it: with Redis stopped, the collectors log queue_publish_failed, still write to the database, and exit normally. No data is lost.

Bad data: why I keep bad NO₂ but drop bad traffic. - A null or repeated NO₂ value is suspicious, not proven wrong. I keep it with is_flagged = TRUE and leave it out of training. - speed = -1 from NDW is not a speed; it means no car was measured. Keeping it would wreck averages (108 and -1 average to 53.5), so I drop that value, log it and count it. The same rule applies to any negative flow, though only speed -1 has appeared.

Consequences. Low coupling, but one more component to run and watch. ADR-004 explains why I removed it.

ADR-003: Resilience

Status: Accepted

Context. AirBreda informs; it controls nothing. An hour of downtime harms nobody, and the sources only update hourly. My database is Single-AZ, which AWS rates at 99.5% availability.

Decision. - **SLO:** 99% availability for `/site/{id}` per 30 days (about 7.2 hours of downtime allowed), and NO₂ never more than 3 hours old. Promising more than the database's own 99.5% would be dishonest. - **Disaster recovery: Backup & Restore.** RDS keeps automatic backups, the code is on GitHub, and every image builds from a Dockerfile. Recovery takes about an hour: new VM, `git clone`, `.env`, `docker build`, restore the snapshot. NO₂ can be fetched again from Luchtmeetnet, so the only data I can't replace is traffic, and that already sits safely in S3. - **Next tier up:** Multi-AZ would add about \$15 a month, roughly doubling the database cost, for availability nobody needs here.

Observability. All containers log structured JSON, including `DATA_QUALITY_ERROR` and `BAD_DATA_THRESHOLD_EXCEEDED` (more than 10 bad values in one run). Each run writes a row to `ingestion_runs`, and `/health` reads it back.

A conflict in the course. Day 2 asks for an in-memory counter and a `/health` endpoint in each collector. Day 3 makes the collectors run for a few seconds and exit, so they can't hold a counter or answer requests. I solved it by having each run save its result to `ingestion_runs`, which the dashboard's `/health` reads.

On CAP. Strictly, CAP is about replicated data, and my database has no replicas. The practical choice is the same, though: if the VM can't reach the database, my collector fails loudly and retries next hour instead of saving data it can't check. I chose consistency over availability for NO₂.

ADR-004: Compute

Status: Accepted (Day 3). Supersedes ADR-002.

Context. The collectors run for seconds per hour (traffic: 6.4 s and 39 MB peak memory). They must run without my laptop. Options: one VM with cron, ECS Fargate with EventBridge Scheduler, or Lambda.

Decision. One **EC2 t3.micro** (2 vCPU, 1 GiB, 8 GB disk, Amazon Linux 2023) with Docker. cron starts both collectors at minute 0 of every hour, and they write straight to RDS and S3. No queue.

Why. - It is the simplest thing that works, and I can see and debug every part over SSH. With hourly jobs and one consumer, a queue adds work and gives nothing back. - The t3.micro fits: 1 GiB is plenty, and "t" instances save up CPU credits while idle and spend them in the short burst at minute 0. A t3.nano (0.5 GiB) would be tight for Docker plus the dashboard. - Cost: about \$7.88 a month for the instance and \$0.67 for the disk. - At 50 corridors with 5-minute

updates I would move to Fargate tasks started by EventBridge Scheduler, with SQS in between. One VM would become a bottleneck and a single point of failure.

Problems I didn't expect. 1. My Free plan blocked Ireland (eu-west-1), so I used Stockholm. It is also in the EU, so GDPR applies the same way. 2. My first S3 write failed with `AccessDenied`: the role had no policies. Instead of granting full S3 access, I wrote a narrow policy (Put/Get on this bucket's files, List on the bucket). The course lists only Put and Get, but the trainer and dashboard also need List. 3. SSH timed out every time I changed networks, because the firewall only allows my current IP. I updated the rule each time. 4. The Free plan's "Express" database setup defaulted to Aurora with public access. I chose plain PostgreSQL, Single-AZ, connected privately to the VM. 5. During the demo, the campus Wi-Fi blocked port 8000; the dashboard only loaded on mobile data. In production I would serve it on port 443 with HTTPS behind a reverse proxy. 6. **I broke collection myself, and nothing told me.** On Friday afternoon, while updating the SSH rule for the campus network, I put my campus IP into the **outbound** rule instead of the inbound one. From then on the VM could only send traffic to that one address, so it could no longer reach Luchtmeetnet, NDW, S3 or the database. Every hourly run from Friday 15:00 to Saturday 16:00 (Dutch time) failed with a connection timeout, and I only found out on Saturday when a retrain hung. Restoring the outbound rule to 0.0.0.0/0 fixed it. Luchtmeetnet re-sends the last 50 hours, so the NO₂ gap filled itself: the first run after the fix logged inserted: 27. But about 26 hours of traffic are lost for good. By my own SLO, that one mistake used more than three months' worth of downtime budget (26 hours against 7.2 per month). The logs and `/health` showed the problem the whole time, but only if someone looked, which is the strongest argument for the alert I describe in the reflection.

ADR-005: Dashboard deployment

Status: Accepted (Day 4). Extends ADR-004.

Context. Day 4 adds a dashboard, which has to run all the time, unlike the collectors.

Decision. The dashboard is a third container on the same VM (FastAPI on port 8000, `--restart unless-stopped`). A separate trainer image, which I run by hand, writes `model.pkl`, and I copy that into the dashboard image when I build it. So there are three services, as in the brief, plus a one-off trainer.

Why not a managed service such as App Runner or ECS? The VM has room to spare and the dashboard has a handful of users. A managed service would add a second deployment path and cost, with nothing a user would notice. I would switch if traffic outgrew one VM or I needed zero-downtime deploys.

Updates. I push to GitHub, then on the VM run `git pull`, `docker build` and restart the container. The model text on the page is read from `model.pkl`, so the page always matches the deployed model.

After a reboot. Docker and cron start at boot and the dashboard restarts itself. I tested this on 3 October by rebooting the VM and checking `docker ps`, `crontab -l` and the page.

What testing caught. Testing the pipeline on test data before deploying caught a timezone bug in `/health` (local time instead of UTC). The trainer and dashboard use the same pinned

library versions to avoid mismatches.

ADR-006: The model and how it is served

Status: Accepted (Day 4). Numbers from my final training run, 2 October 12:27.

Context. An hour counts for training only if it has an unflagged NO₂ value and traffic from all four sensors. Traffic collection started on Thursday evening and NDW keeps no history, so I had 16 usable hours.

Matching the two sources in time. Everything is stored in UTC. A Luchtmeetnet value labelled 10:00 is the average for 09:00 to 10:00. The 10:00 traffic run captures the minute 09:59, which I round to the nearest hour. UTC also avoids summer-time problems: on 25 October the hour from 02:00 to 03:00 happens twice in Dutch time, but never in UTC.

Decision. - Model: linear regression on two features: total vehicles per hour across the four sensors (each sensor's lanes added up) and hour of day in UTC. With this little data, anything more complex would memorise noise. The brief asks for it, and the result can be explained in one sentence. - **Results** (in-sample; with fewer than 20 rows I couldn't hold back a test set):

Run	Rows	R ²	Adjusted R ²	MAE (µg/m ³)	Traffic effect per 1,000 veh/h
07:52	11	0.43	≈ 0.29	4.48	−3.83
09:57	13	0.07	≈ −0.11	5.18	+0.22
12:27 (deployed)	16	0.10	≈ −0.04	4.50	+0.82

The deployed formula: NO₂ = 27.4 + 0.00082 × vehicles per hour + 0.19 × hour (UTC). For 3,000 vehicles at 08:00 UTC that gives about 31.4 µg/m³.

- **What the results mean.** The estimate is unstable. Two extra hours flipped the direction of the traffic effect and R² dropped from 0.43 to 0.07. Adjusted R² is negative, so the model does no better than guessing the average. That doesn't prove traffic has no effect; it shows 16 mostly night-time hours can't tell the direction yet. Likely reasons: a one-minute traffic snapshot against a full-hour NO₂ average, almost no rush hours or weekends in the data, and the hour treated as a plain number, so 23:00 and 00:00 look far apart.
- **Risk score:** a sigmoid around 40 µg/m³, the EU annual limit: 50% at 40, near 0 well below. Because 40 is an annual limit, the score means "this looks like a high hour", not a legal breach. The EU hourly limit is 200, but every hour I measured was between 10 and 41, so a 200 threshold would always show 0 and say nothing. From 2030 the annual limit drops to 20 (Directive (EU) 2024/2881).
- **Serving:** the dashboard imports `predict.py` and the model is baked into the image. A model this small doesn't need its own server.

Training-serving skew. The model was trained on the total of all four sensors, so every `/site/{id}` call passes that same total, not the sensor's own value. That is why all four sites show the same prediction: there is one air station, so there is one NO₂ value to predict. Training and serving both use the UTC hour and the same library versions.

Why one station for all four sensors. The sensors sit within about 100 m of each other at one junction. Luchtmeetnet lists NL10240 as a traffic station, about 330 m from them by my calculation.

When the model fails. `/site/{id}` still returns the measured NO₂ and traffic, with the prediction set to null and an error message. I tested this by removing the model file. A broken model shouldn't hide real data.

The dashboard. It shows a sentence rewritten every hour from live data, the current measured and expected NO₂ and traffic, an hour-by-hour chart, a scatter chart of traffic against NO₂ with the model's line, a map of the junction, the model in plain words, and the health of both collectors. The page uses the same API as everyone else, so the page and the API can't disagree.

3. Trade-offs

Storage. I compared DynamoDB, a database on its own, and PostgreSQL plus S3. I chose PostgreSQL for structured NO₂ rows (about 8,760 a year) and S3 for files at \$0.023 per GB-month. I gave up DynamoDB's scaling, which I don't need, and accepted that traffic lives in files rather than a table.

Compute. I compared Lambda, Fargate and one VM. I chose one t3.micro for about \$8.55 a month: full control and easy debugging. I gave up automatic scaling, managed patching and fault tolerance. The VM is a single point of failure, which my Backup & Restore plan accepts.

Messaging. I compared Redis (built on Day 2), SQS and direct writes. I chose direct writes, because with one consumer and hourly jobs a broker is cost without benefit. I gave up decoupling, so adding a consumer means changing the collectors.

Disaster recovery. I compared Backup & Restore, Pilot Light, Warm Standby and Multi-AZ. I chose Backup & Restore, with about an hour to recover. Multi-AZ would roughly double the database cost for availability an information dashboard doesn't need.

4. Cloud provider rationale (for a policy officer at the Municipality of Breda)

I chose Amazon Web Services (AWS) to run AirBreda. In plain terms, AWS rents me three things: a small computer that runs the programs day and night, a secure database for the air-quality measurements, and storage for the original traffic files. You pay only for what you use, which for this pilot is about 23 dollars a month.

Is that appropriate for a Dutch public body? First, location. All data is stored in AWS's Stockholm data centre, inside the European Union, so European privacy law (GDPR) applies. AWS's centres in Ireland or Frankfurt would work just as well. Second, the data itself is low-risk. It is public measurement data from RIVM and the national traffic portal NDW, with no personal information. Third, access is tightly controlled. The database can't be reached from the internet at all, only by the project's own computer, and that computer can only write to one storage space. No passwords are stored in the code.

What would the municipality lose by moving to another provider, such as Microsoft Azure? Very little of the actual work. The programs run in standard containers that behave the same on any cloud, and the database is standard PostgreSQL. A move would mean rebuilding the cloud setup, perhaps a day's work, and checking the security settings again. The programs and the data would move with it. I chose this on purpose, so the municipality isn't tied to one supplier.

The main risk isn't the technology but the cost. Cloud bills grow quietly when nobody watches them. For a production service I would set a budget alert, add automatic backups to a second location, and review the costs once a month.

(about 280 words)

5. Cost estimate (AWS Stockholm, on-demand, USD per month)

Prices from the AWS price list for Stockholm (1 October 2026): EC2 t3.micro \$0.0108/h; RDS db.t4g.micro Single-AZ \$0.016/h; RDS storage \$0.12/GB-month; EBS gp3 \$0.0836/GB-month; S3 \$0.023/GB-month and \$0.005 per 1,000 uploads. A month is 730 hours.

Component	1 corridor (now)	10 corridors	50 corridors
VM + disk	t3.micro + 8 GB: \$8.55	same: \$8.55	t3.small + 16 GB: \$17.11
Database	db.t4g.micro + 20 GB: \$11.68 + \$2.40 = \$14.08	same: \$14.08	Multi-AZ + 20 GB: \$28.89
S3	about 0.9 GB + 3,650 uploads: \$0.04	0.9 GB + 29,900 uploads: \$0.17	0.9 GB + 146,700 uploads: \$0.75
Total	≈ \$22.67	≈ \$22.80	≈ \$46.75

Assumptions. A corridor is one air station plus four traffic sensors. The NDW file is national, so I download it once an hour however many corridors there are; extra corridors mostly add uploads of small CSV files. Storage grows month by month; the table shows the first month. My account runs on Free plan credits, so my actual bill is \$0.

Does one VM still work? At 10 corridors, yes: it is still seconds of work per hour. At 50 corridors with 5-minute updates, no. I would move to Fargate, EventBridge Scheduler and SQS, and make the database Multi-AZ because more people would depend on it.

6. Reflection

The decision I'm least sure about is storing the traffic only as CSV files in S3. Keeping the raw NDW files there was right, because that data can never be downloaded again. But the dashboard now has to list the bucket to find the newest files. With four sensors that's quick; with 50 corridors it would be slow and cost more. To find out, I would measure the dashboard's response time under load and compare it with a small traffic table in PostgreSQL.

The weakest part of my data is the traffic itself. NDW only publishes the current minute, so every hourly traffic value is a one-minute snapshot, while the NO₂ value is a full-hour average. I'm comparing one minute with sixty. That showed up clearly in my model. At 07:52, with 11 hours, traffic seemed to lower NO₂. Two hours later it seemed to raise it slightly, and R² fell

from 0.43 to 0.07. By the final run, with 16 hours, adjusted R^2 was still negative. That instability taught me more than any single number: the data can't answer the question yet, and what I've actually delivered is the pipeline, not the conclusion.

With a full year of data I would change three things. First, the features: weekday or weekend, holidays, wind, temperature, the previous hours' NO_2 , and the hour as a point on a clock, so that 23:00 and 00:00 sit next to each other. Second, the traffic: fetch every 5 or 15 minutes and average them into a real hourly value. Third, the testing: train on the first nine months, test on the last three, and look closely at the error in rush hours. I would only switch to a more complex model if it beat the linear one on that test.

The first thing I'd add for production is Infrastructure as Code with Terraform, plus a CI/CD pipeline. I created every resource by clicking through the AWS console, and most of my problems this week came from that: the wrong default region, a role without permissions, and a firewall rule tied to an IP address that changed whenever I switched networks. In code, those settings would be visible, reviewable and repeatable. The second addition would be an alert when `/health` shows stale data. I learned that the hard way: a firewall rule I changed by mistake stopped all collection for about 26 hours, and I only noticed a day later.

If I did this project again, I would start collecting data on the first day. My model was limited by the number of hours I had, and every hour I didn't collect is gone for good, because NDW keeps no history. I also learned that the course's Day 2 and Day 3 instructions contradicted each other, and that working out a solution, the `ingestion_runs` table, taught me more than following either one to the letter. Finally, the demo showed me that a network I don't control can block a working system. The dashboard loaded on mobile data but not on the campus Wi-Fi, which is a good argument for serving it on port 443 with HTTPS.

(about 520 words)